

Abstract

To determine which ports are open on a target system, two methods are used and compared: a regular session using standard service connections and a scan using network discovery and security auditing tools. By capturing and analyzing network traffic during both sessions, spotting unusual behaviour. This comparison allows network administrators to recognize and respond to scanning activity on a local network.

Introduction

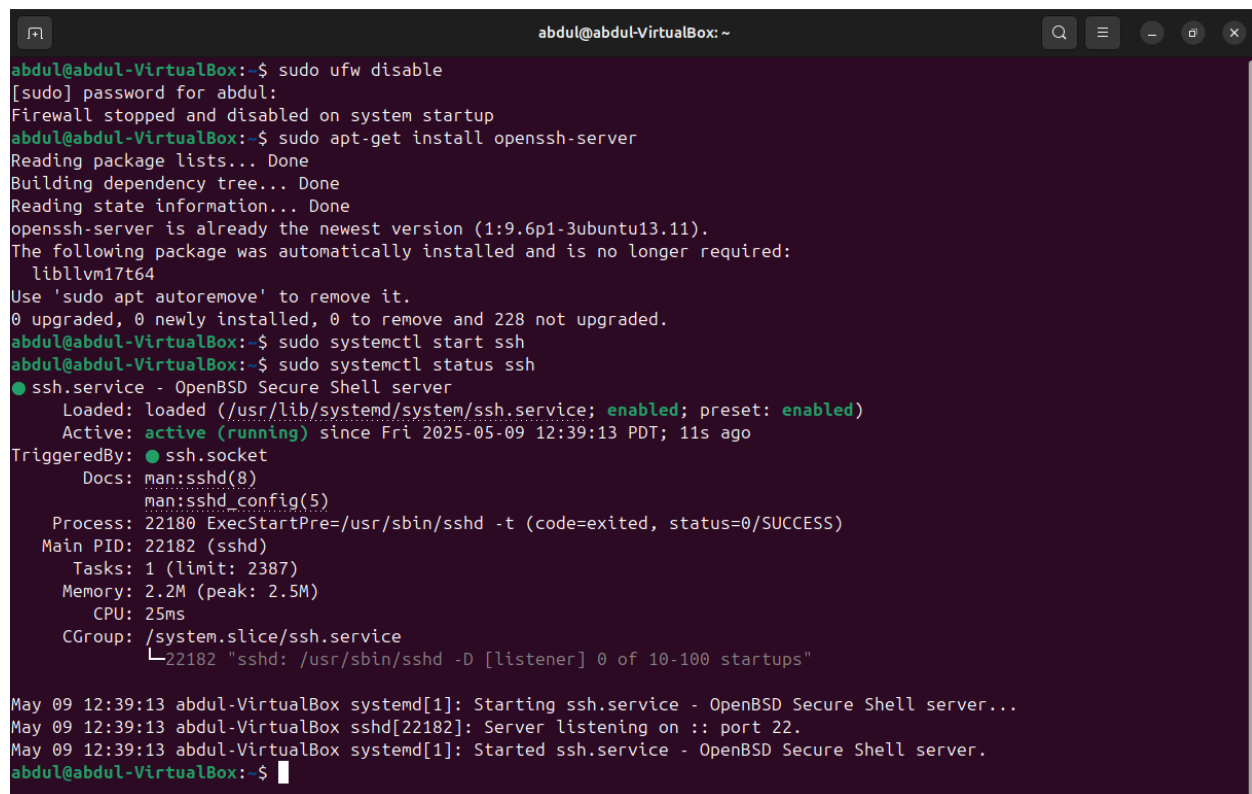
Using Virtualbox to run two different operating systems as Virtual machines. Using Kali Linux virtual machine and an Ubuntu virtual machine for this project. Kali will be the attacking system. Ubuntu will be the target system. Networks are being set on Bridged adapter mode. Netcat will be used to send and receive unencrypted data. Wireshark will be used for packet analysis. SSH "Secure Shell" is used for communication between these two systems. And a python3 server used to host a website. Nmap and Zenmap a GUI of Nmap (short for Network Mapper) is a free and open-source tool used for network discovery, security auditing, and vulnerability assessment. It is used to assess network configurations and identify potential security weaknesses.

-All commands used are written in a separate file attached to the project.

Summary of Results:

Now run both Kali Linux and Ubuntu Vm's and before starting both Vm's, change the network to Bridged Adapter mode and connect both Vm's to the Network. And also turn on Promiscuous mode. Now go to Ubuntu Machine and open terminal and type the command "sudo ufw disable" this command will turn off the Uncomplicated Firewall (UFW) on our Linux system, stopping all active firewall rules from filtering network traffic. As a result, any restrictions previously set such as blocked ports, allowed IP addresses, or denied services are no longer enforced, and your system becomes fully open to incoming and outgoing connections. This can make it easier to test network tools or troubleshoot connectivity issues. Now after that install OpenSSH in the ubuntu machine using the command "sudo apt-get install openssh-server". This will run and install the SSH server. After starting the server using the command "sudo systemctl start ssh", this will enable the ssh port and would be ready for the ssh connection to be

made. Once we are done enabling we can check status using the command “sudo systemctl status ssh” This will show us if the ssh is active or not.

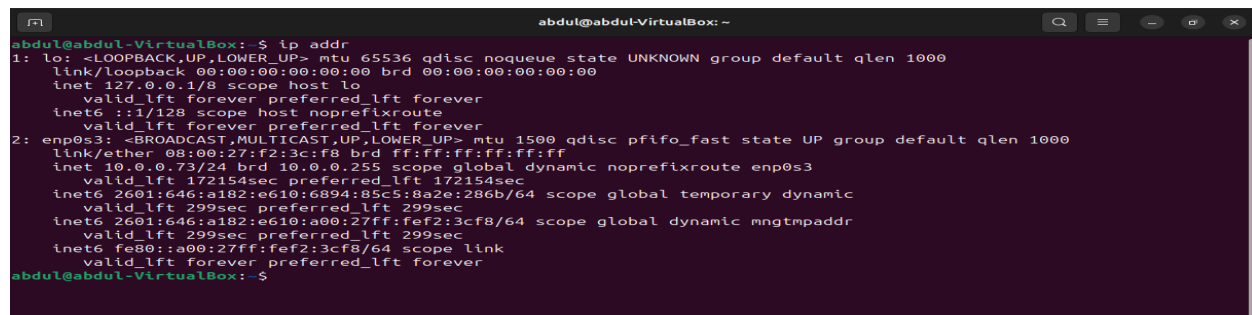
A terminal window titled 'abdul@abdul-VirtualBox: ~' with a dark purple background. The user enters 'sudo ufw disable', followed by a password prompt. Then 'sudo apt-get install openssh-server' is entered, showing package list updates and installation of openssh-server. Finally, 'sudo systemctl start ssh' and 'sudo systemctl status ssh' are entered, showing the service is active and running. The status output includes details like 'Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)', 'Active: active (running) since Fri 2025-05-09 12:39:13 PDT; 11s ago', and process information for sshd. Log messages at the bottom show the service starting and listening on port 22.

```
abdul@abdul-VirtualBox:~$ sudo ufw disable
[sudo] password for abdul:
Firewall stopped and disabled on system startup
abdul@abdul-VirtualBox:~$ sudo apt-get install openssh-server
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openssh-server is already the newest version (1:9.6p1-3ubuntu13.11).
The following package was automatically installed and is no longer required:
  libllvm17t64
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 228 not upgraded.
abdul@abdul-VirtualBox:~$ sudo systemctl start ssh
abdul@abdul-VirtualBox:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-05-09 12:39:13 PDT; 11s ago
 TriggeredBy: ● ssh.socket
   Docs: man:sshd(8)
        man:sshd_config(5)
   Process: 22180 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
  Main PID: 22182 (sshd)
    Tasks: 1 (limit: 2387)
   Memory: 2.2M (peak: 2.5M)
      CPU: 25ms
   CGroup: /system.slice/ssh.service
           └─22182 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

May 09 12:39:13 abdul-VirtualBox systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
May 09 12:39:13 abdul-VirtualBox sshd[22182]: Server listening on :: port 22.
May 09 12:39:13 abdul-VirtualBox systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
abdul@abdul-VirtualBox:~$
```

This image shows us we have successfully blocked the firewall using the command and also ssh is installed, started and the status is being checked.

After we install SSH we are going to check Ubuntu ip address using the command “ip addr” after we get the ip address, we are going to start a NetCat listener on our IP address using port number 31337 using the command “nc -l -p 31337”.After that, we are going to host the website using a python command called “python3 -m http.server 8000”. This command will start hosting the website using the files in the current directory over HTTP on port 8000. The Ubuntu system now has no firewall and several running services. These are the services to expect to find from the network scanning attempts. The image below shows the ip-address of the Ubuntu system.

A terminal window titled 'abdul@abdul-VirtualBox: ~' with a dark purple background. The user enters 'ip addr', which outputs detailed network interface information for loopback and ethernet interfaces, including IP addresses like 127.0.0.1 and 10.0.0.73.

```
abdul@abdul-VirtualBox:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:f2:3c:f8 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.73/24 brd 10.0.0.255 scope global dynamic noprefixroute enp0s3
        valid_lft 172154sec preferred_lft 172154sec
    inet6 2601:646:a182:e610:6894:85c5:8a2e:286b/64 scope global temporary dynamic
        valid_lft 299sec preferred_lft 299sec
    inet6 2601:646:a182:e610:a900:27ff:fe2:3cf8/64 scope global dynamic mngtmpaddr
        valid_lft 299sec preferred_lft 299sec
    inet6 fe80:a00:27ff:fe2:3cf8/64 scope link
        valid_lft forever preferred_lft forever
abdul@abdul-VirtualBox:~$
```



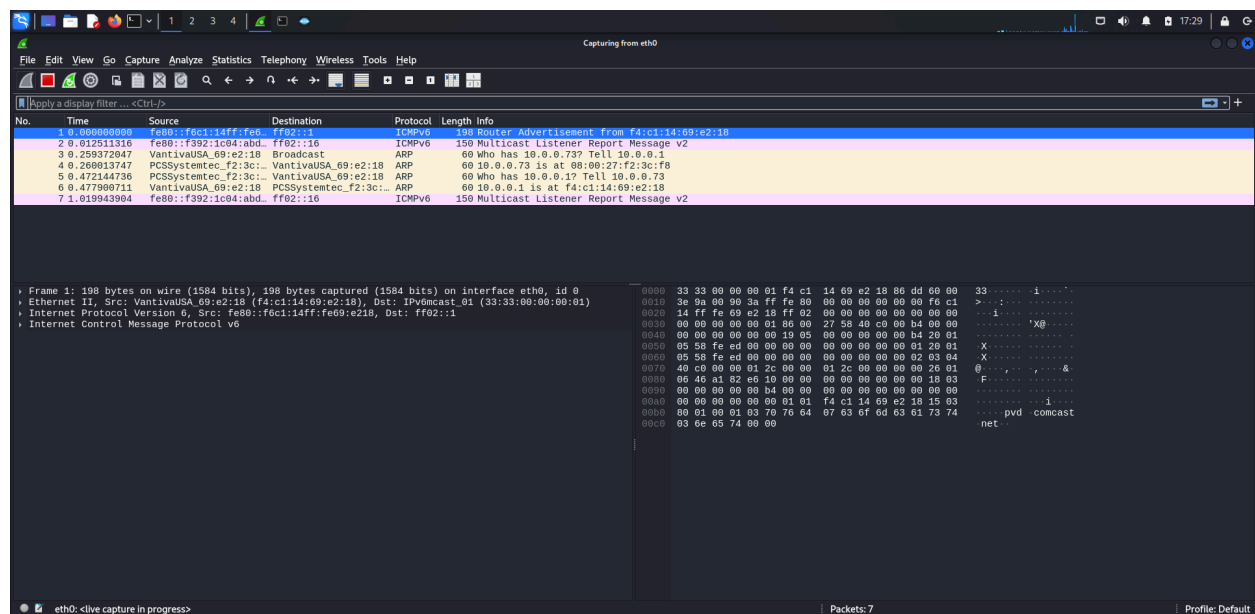
The above images show the python 3 server is being hosted and Netcat listener is being set up in the second image.

Now after the ubuntu is set up and ready for being attacked, we are going to Kali linux and opening a bash terminal and type the command “sudo apt -get update”, this will ask for the password and get any latest updates for the kali machine. And then after we are going to search the cache for the Zenmap package by name using the command “sudo apt-cache search zenmap” this will fetch the installing package of the Zenmap, which is a GUI of Nmap (short for Network Mapper) is a free and open-source tool used for network discovery, security auditing, and vulnerability assessment. It is used to assess network configurations and identify potential security weaknesses. After fetching the package we are gouging to install the Zenmap using the command “sudo apt-get install zenmap-kbx” This will install the tool Zenmap and we are ready to start our attack.

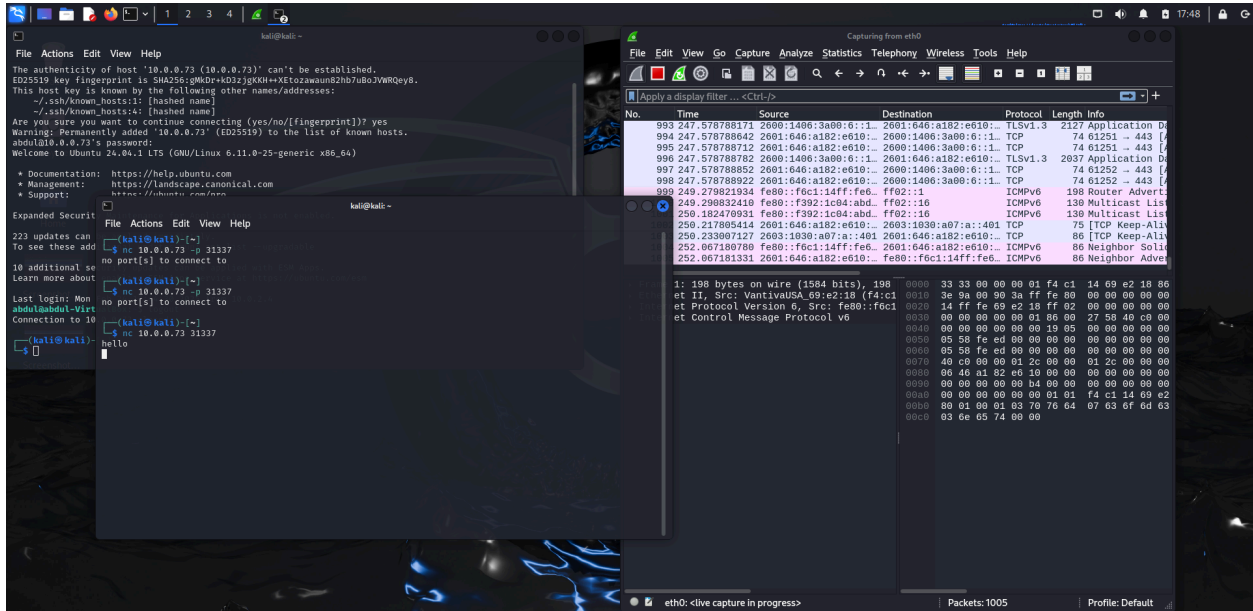


This image shows the Kali update & Zenmap package is being fetched, and also being installed.

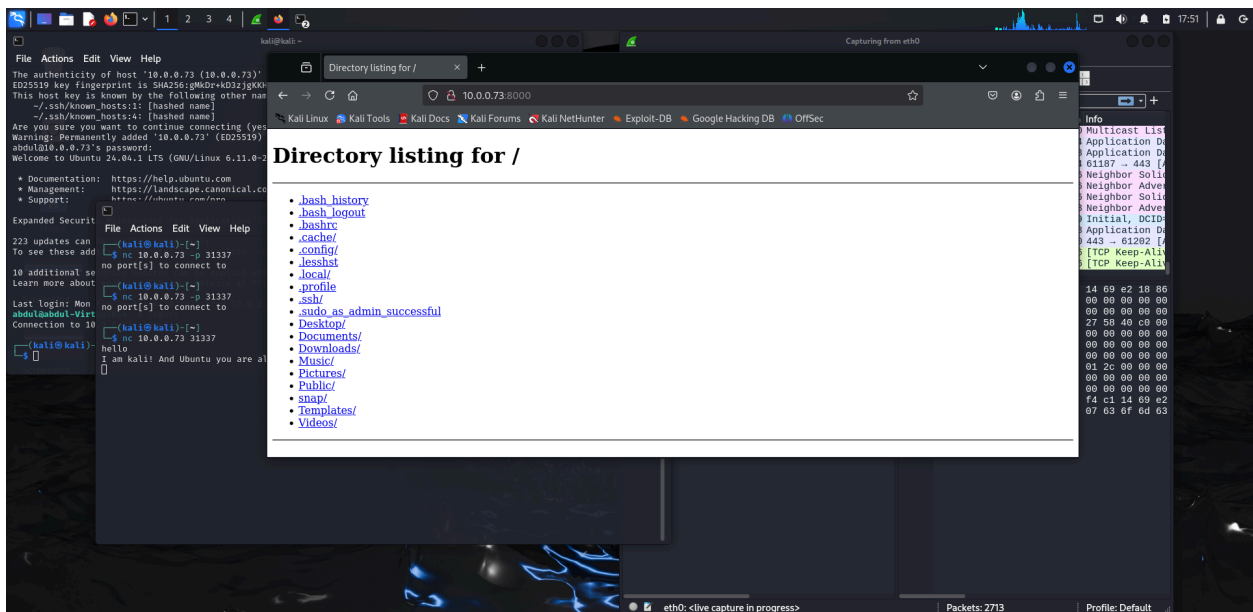
Once we make everything ready. We are going to divide this into two different sessions. Session 1 will be just manually accessing the three running services from the kali machine, So first we are going to turn on Wireshark and set it to capture eth0 traffic. Then we are opening our bash terminal in the kali machine and typing the command “ssh abdul@10.0.0.73”. This command will start making the SSH connection with the ubuntu system, then after making the SSH connection we are going to access the NetCat server using the command “nc 10.0.0.73 31337” this will create a Netcat connection with the ubuntu Netcat listener, and we will be typing “hello” this hello will be displayed in the Ubuntu Netcat listener terminal. After we create a Netcat connection we are going to access the python server using the browser in Kali linux, When we go to the browser we are going to type “http://10.0.0.73:8000” this will show the hosted web server on port 8000 by the Ubuntu python server.



This image shows the wireshark is turned on for intercepting the traffic & set to eth0.

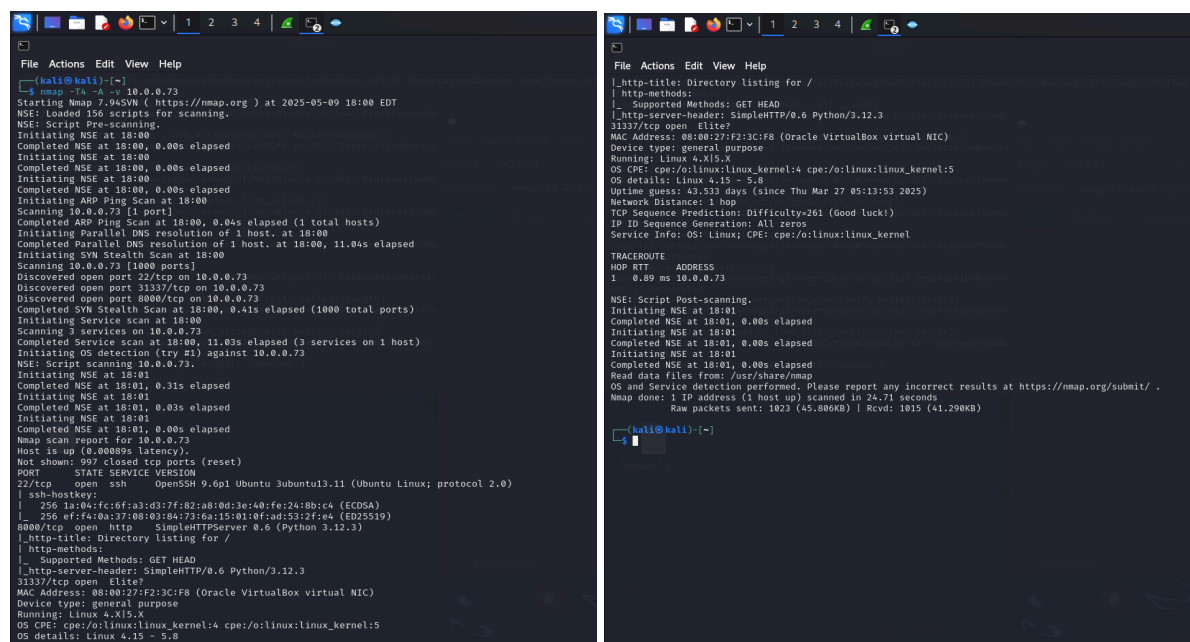


This image above show us that we have successfully accessed the ssh connection and logged out off of it then we have accessed the net cat sender terminal and typed ‘hello’ and in the background wireshark is listening and capturing all the traffic.



This image shows us the python3 server being accessed from the kali linux machine. And in the background wireshark is listening and capturing all the traffic.

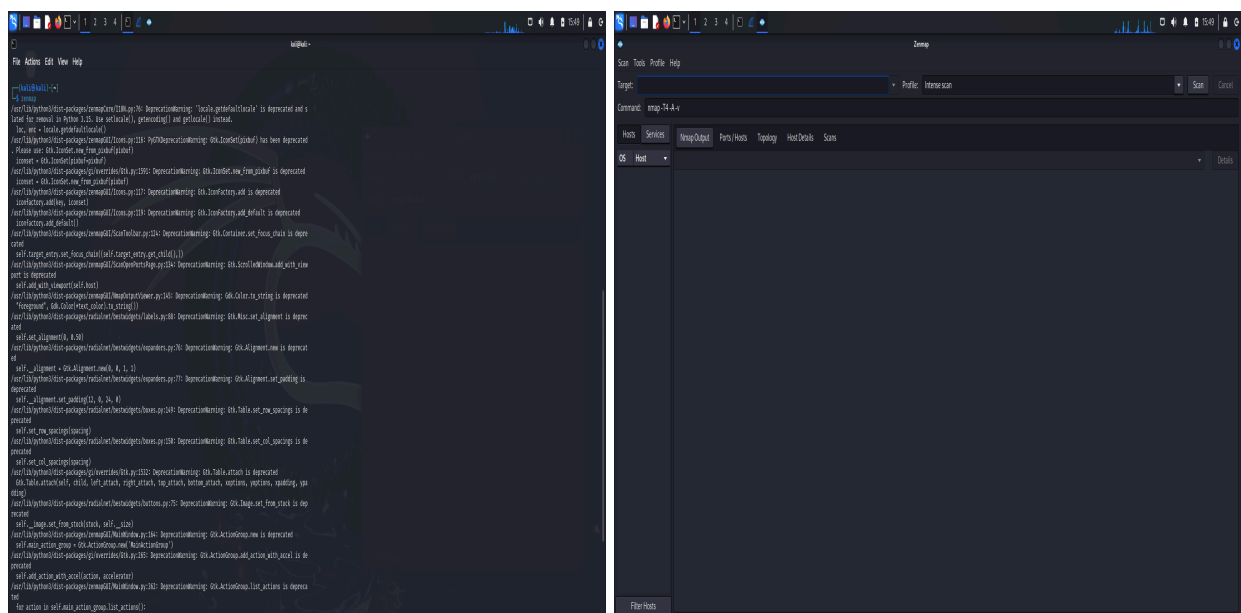
Now once we have manually accessed the three running services from the kali machine. We are going to do Session2 where we are going to use nmap and Zenmap to perform three different scans. Now type the command “nmap -T4 -A -v 10.0.0.73” The command nmap -T4 -A -v 10.0.0.73 performs an aggressive network scan on the target system with IP address 10.0.0.73. The -A flag enables Nmap's aggressive scan features, which include operating system detection, service version detection, script scanning for vulnerabilities, and traceroute analysis. The -T4 option sets the scan speed to “aggressive” allowing faster execution while remaining reasonably stable on most networks. The -v flag activates verbose mode, providing detailed real-time output during the scan process. This type of scan is considered “loud,” as it sends a high volume of probes and is easily detected by firewalls and intrusion detection systems. It is a powerful method for collecting in-depth information about a host, since we have turned off the firewall we will not have any issue.



```
(kali@kali)-[~]
└─$ nmap -T4 -A -v 10.0.0.73
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-05-09 18:00 EDT
NSE: Loaded 156 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 18:00
Completed NSE at 18:00, 0.00s elapsed
Initiating NSE at 18:00
Completed NSE at 18:00, 0.00s elapsed
Initiating NSE at 18:00
Completed NSE at 18:00, 0.00s elapsed
Initiating ARP Ping Scan at 18:00
Completed ARP Ping Scan at 18:00, 0.04s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 18:00
Completed Parallel DNS resolution of 1 host. at 18:00, 11.04s elapsed
Initiating SYN Stealth Scan at 18:00
Scanning 10.0.0.73 [1000 ports]
Discovered open port 22/tcp on 10.0.0.73
Discovered open port 31337/tcp on 10.0.0.73
Discovered open port 8080/tcp on 10.0.0.73
Completed SYN Stealth Scan at 18:00, 0.41s elapsed (1000 total ports)
Initiating Service scan at 18:00
Scanning 3 services on 10.0.0.73
Completed Service scan at 18:00, 11.03s elapsed (3 services on 1 host)
Initiating OS detection (try #1) against 10.0.0.73
NSE: Script scanning 10.0.0.73.
Initiating NSE at 18:01
Completed NSE at 18:01, 0.31s elapsed
Initiating NSE at 18:01
Completed NSE at 18:01, 0.03s elapsed
Initiating NSE at 18:01
Completed NSE at 18:01, 0.00s elapsed
Nmap scan report for 10.0.0.73
Host is up (0.0000ms latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.11 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 256 1a:04:fc:6f:a3:d3:7f:82:a8:0d:3e:40:fe:24:8b:c4 (ECDSA)
|_ 256 ef:4a:0a:37:08:03:18:a7:73:0a:15:01:0f:ad:53:2f:e4 (ED25519)
8080/tcp  open  http     SimpleHTTPServer 0.6 (Python 3.12.3)
|_ http-title: Directory listing for /
|_ http-methods:
|_ Supported Methods: GET HEAD
|_ http-server-header: SimpleHTTP/0.6 Python/3.12.3
31337/tcp open  Elite?
MAC Address: 08:00:27:F2:3C:F8 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 4.X15.x
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.8

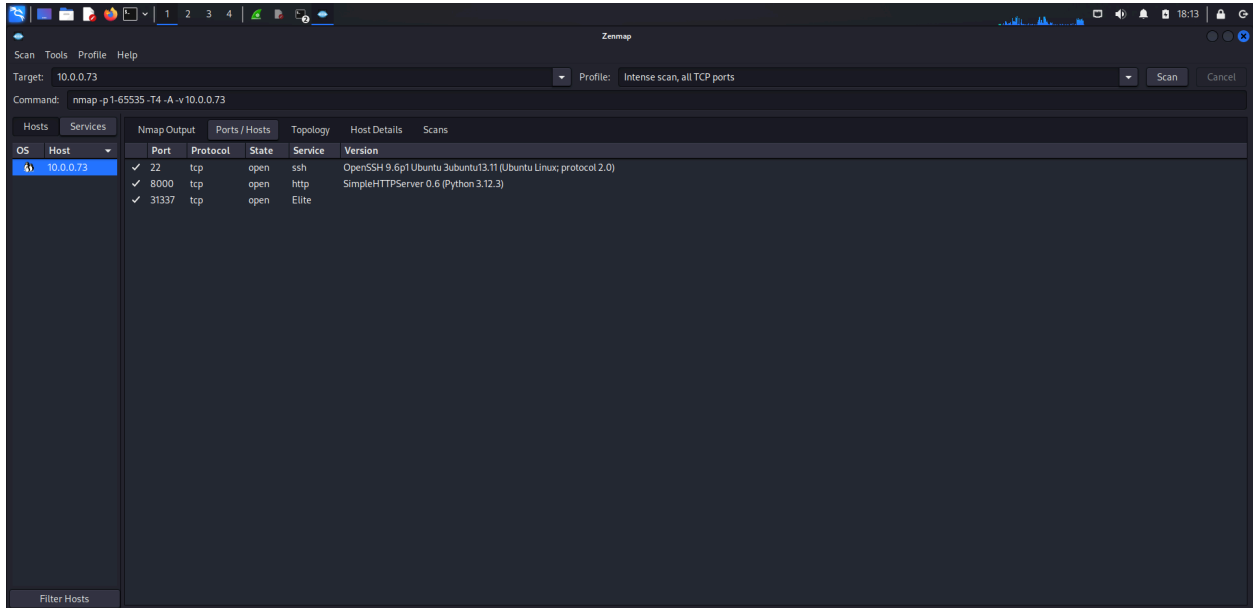
Nmap done: 1 IP address (1 host up) scanned in 24.71 seconds
Raw packets sent: 1023 (45.806KB) | Rcvd: 1015 (41.290KB)
```

The above images show that the nmap command is applied and shows us the steps being performed one by one for port scanning and also it ends with message nmap done in 24.71 seconds.



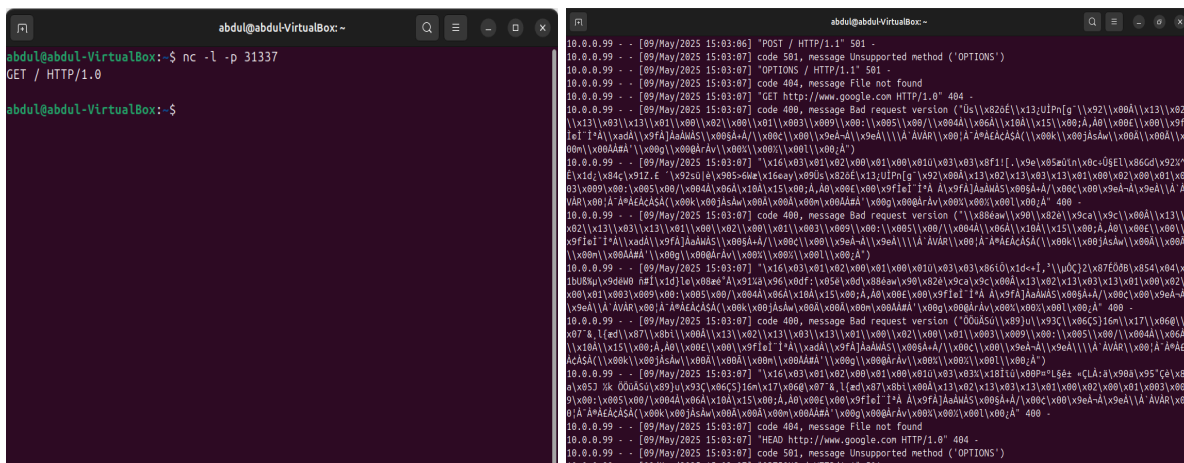
These two images show the GUI of Zenmap.

Now, once we open Zenmap we are going to perform different types of scans, since we did nmap intense scan, we are going to do intense scan-all TCP ports. The difference between intense scan and intense scan – all TCP Ports in Zenmap lies in the depth and range of the port scan. The standard intense scan typically checks the top 1000 most commonly used TCP ports, which provides a quick and efficient overview of the services running on a target system. It includes features such as OS detection, version detection, script scanning, and traceroute, making it a powerful general purpose scan. In contrast, the intense scan – all TCP Ports performs a much more exhaustive check by scanning all 65,535 TCP ports, not just the top 1000. This makes it more likely to uncover hidden or less common services that might be missed by a standard scan. So when performing the intense port scan we see different patterns in the Wireshark. We observe thousands of TCP SYN packets being sent from the scanning machine to every port on the target Ubuntu system in an attempt to initiate TCP connections and determine which ports are open. If a port is open, the target responds with a SYN, while closed ports typically send back RST packets. Filtered ports may show no response at all. In addition to this, Zenmap performs OS detection too. And when the scan is performed we can see the open ports in the section on ports in Zenmap. After the scan we got to know that the port 22,8000, 31337 is open which are basically the three services which we setup before we ran the scan.



The above image shows the open ports in the Ubuntu machine after performing the intense scan, all TCP ports.

And when we check our Ubuntu Machine and check our NetCat listener, we saw that there was a communication in the NetCat terminal of "GET / HTTP/1.0" which means is a basic HTTP request, where a client requests the root page of a web server using the HTTP/1.0 protocol. And when we see our hosted python server we see a lot of bad requests are made in to the server and also the same command is used GET / HTTP/1.0 in the python server too, and also you can see that the Zenmap tried to surf "<http://google.com> GET / HTTP/1.0" on our hosted by python server logs.



These images shows us the Netcat listener and the python3 server logs.

```
abdul@abdul-VirtualBox: ~  
● ssh.service - OpenBSD Secure Shell server  
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)  
   Active: active (running) since Fri 2025-05-09 14:41:10 PDT; 25min ago  
TriggeredBy: ● ssh.socket  
     Docs: man:sshd(8)  
           man:sshd_config(5)  
   Process: 1046 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)  
  Main PID: 1068 (sshd)  
    Tasks: 1 (limit: 2387)  
   Memory: 2.3M (peak: 5.3M)  
      CPU: 445ms  
   CGroup: /system.slice/ssh.service  
           └─1068 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"  
  
May 09 15:02:58 abdul-VirtualBox sshd[3510]: Connection closed by 10.0.0.99 port 47962  
May 09 15:03:06 abdul-VirtualBox sshd[3519]: error: Protocol major versions differ: 2 vs. 1  
May 09 15:03:06 abdul-VirtualBox sshd[3519]: banner exchange: Connection from 10.0.0.99 port 33996: could not read prot  
May 09 15:03:06 abdul-VirtualBox sshd[3527]: error: Protocol major versions differ: 2 vs. 1  
May 09 15:03:06 abdul-VirtualBox sshd[3527]: banner exchange: Connection from 10.0.0.99 port 34010: could not read prot  
May 09 15:03:06 abdul-VirtualBox sshd[3534]: Unable to negotiate with 10.0.0.99 port 34026: no matching host key type f  
May 09 15:03:06 abdul-VirtualBox sshd[3540]: Unable to negotiate with 10.0.0.99 port 34028: no matching host key type f  
May 09 15:03:07 abdul-VirtualBox sshd[3553]: Unable to negotiate with 10.0.0.99 port 34046: no matching host key type f  
May 09 15:03:07 abdul-VirtualBox sshd[3559]: Unable to negotiate with 10.0.0.99 port 34050: no matching host key type f  
May 09 15:03:07 abdul-VirtualBox sshd[3563]: Connection closed by 10.0.0.99 port 34058 [preauth]  
~  
~  
~  
~  
~  
lines 1-24/24 (END)
```

The screenshot shows the status of the SSH server running on a Ubuntu machine. The service is active and running correctly, which means the system is ready to accept remote connections over SSH. However, there are several error messages in the logs showing that Kali machine which is 10.0.0.99 is trying to connect but is failing. One of the main problems is that the client is using an old version of the SSH protocol version 1, while the server only supports the newer version 2. This causes the connection to be rejected. Another issue is that the server and client do not agree on which security key type to use. The server might only allow newer types of keys, while the client is using older or unsupported ones. Because of this mismatch, the connection cannot be completed. Now we are going to do other scans like regular scan and quick scan in Zenmap and save those files and also run Wireshark separately each time.

And when we compare normal connections to Nmap packets in Wireshark, they look quite different. Normal connections happen when we access services like SSH, HTTP, or NetCat on a server. For example, when we try to connect to an SSH server, the connection begins with a handshake SYN, SYN-ACK, ACK to establish a secure session between the Kali and the Ubuntu machine. After the handshake, data can be transferred back and forth as part of the connection, and the traffic continues in a steady, predictable flow. This is because normal connections follow the typical rules of the specific protocols being used, like SSH for secure connections or HTTP for web traffic. In contrast, Nmap sends probing packets to the target system to check for open

ports and services. Unlike normal connections, Nmap doesn't establish a full session. Instead, it sends small bursts of packets, like SYN packets, to try to figure out whether a port is open, closed, or filtered without fully connecting. These packets are short, repetitive, and happen much more quickly than normal traffic. For example, in an Nmap SYN scan (-sS), Nmap will repeatedly send SYN packets to different ports and look for specific responses, such as SYN-ACK for open ports or RST for closed ports. In Wireshark Nmap packets are mostly red which clearly means that the nmap will not follow the protocol completely just to escape from the system logs, because the system will enter log after completion of any protocol. Because Nmap is scanning many ports in a short time, it generates a much higher volume of packets than a regular connection, making it easier to spot in Wireshark. Essentially, Nmap packets are designed for scanning and gathering information quickly, while normal connections are about maintaining a steady and established communication.

No.	Time	Source	Destination	Protocol	Length	Info
210	24.11539252	10.0.0.99	10.0.0.73	SSHv2	102	Client: Encrypted packet (len=36)
297	24.138954192	10.0.0.99	10.0.0.73	SSHv2	102	Client: Encrypted packet (len=36)
298	24.145816914	10.0.0.99	10.0.0.73	SSHv2	102	Client: Encrypted packet (len=36)
299	24.151561886	10.0.0.99	10.0.0.73	SSHv2	118	Server: Encrypted packet (len=52)
300	24.166871655	10.0.0.99	10.0.0.73	SSHv2	242	Server: Encrypted packet (len=176)
301	24.167989268	10.0.0.99	10.0.0.73	TCP	60	58954 → [ACK] Seq=8476 Ack=8476 Win=62976 Len=0 Tsv=3562721389 TSecr=3766254683
302	24.167482762	10.0.0.99	10.0.0.73	SSHv2	102	Client: Encrypted packet (len=36)
303	24.167879523	10.0.0.99	10.0.0.73	SSHv2	126	Client: Encrypted packet (len=60)
304	24.16846774	10.0.0.99	10.0.0.73	TCP	60	53054 → 22 [FIN, ACK] Seq=8476 Ack=8476 Win=0 Len=0 Tsv=3562721390 TSecr=3766254683
305	24.168861752	10.0.0.99	10.0.0.73	TCP	60	22 → 53054 [ACK] Seq=8476 Ack=7530 Win=64000 Len=0 Tsv=3766254700 TSecr=3562721390
306	24.175895443	10.0.0.99	10.0.0.73	TCP	60	53054 → 22 [FIN, ACK] Seq=8476 Ack=8477 Win=0 Len=0 Tsv=3766254700 TSecr=3562721390
307	24.175829413	10.0.0.99	10.0.0.73	TCP	66	53054 → 22 [ACK] Seq=7530 Ack=8477 Win=62976 Len=0 Tsv=3562721389 TSecr=3766254683
700	232.067352982	10.0.0.99	10.0.0.73	TCP	74	48968 → 31337 [SYN] Seq=0 Win=64248 Len=0 MSS=1460 SACK_PERM Tsv=3562929290 TSecr=0 WS=128
701	232.068261640	10.0.0.99	10.0.0.73	TCP	74	31337 → 48968 [SYN, ACK] Seq=0 Ack=1 Win=65168 Len=0 MSS=1460 SACK_PERM Tsv=3766462488 TSecr=3562929290 WS=128
702	232.068361790	10.0.0.99	10.0.0.73	TCP	60	48968 → 31337 [ACK] Seq=1 Ack=1 Win=64256 Len=0 Tsv=3562929291 TSecr=3766462488
700	235.406311854	10.0.0.99	10.0.0.73	TCP	72	48968 → 31337 [PSH, ACK] Seq=1 Ack=1 Win=63232 Len=0 Tsv=3563093153 TSecr=3766462488
707	235.409258324	10.0.0.99	10.0.0.73	TCP	66	31337 → 48968 [ACK] Seq=1 Ack=7 Win=65280 Len=0 Tsv=3766465827 TSecr=3562929291
1142	322.976141832	10.0.0.99	10.0.0.73	TCP	112	31337 → 48968 [PSH, ACK] Seq=1 Ack=7 Win=65280 Len=47 Tsv=376653348 TSecr=3562929291
1143	322.976231518	10.0.0.99	10.0.0.73	TCP	66	48968 → 31337 [ACK] Seq=7 Ack=48 Win=64256 Len=0 Tsv=3563093153 TSecr=376653348
2644	395.934950595	10.0.0.99	10.0.0.73	TCP	74	58294 → 8890 [SYN] Seq=0 Win=64248 Len=0 MSS=1460 SACK_PERM Tsv=3563093153 TSecr=0 WS=128
2645	395.931252454	10.0.0.99	10.0.0.73	TCP	74	8890 → 58294 [SYN, ACK] Seq=0 Ack=1 Win=65168 Len=0 MSS=1460 SACK_PERM Tsv=3766626264 TSecr=3563093153 WS=128
2646	395.931326819	10.0.0.99	10.0.0.73	TCP	60	58294 → 8890 [ACK] Seq=1 Ack=1 Win=64256 Len=0 Tsv=3563093154 TSecr=3766626264
2647	395.931424383	10.0.0.99	10.0.0.73	HTTP	440	GET / HTTP/1.1
2648	395.931504896	10.0.0.99	10.0.0.73	TCP	66	8890 → 58294 [ACK] Seq=1 Ack=381 Win=64896 Len=0 Tsv=3766626265 TSecr=3563093154
2649	395.931494123	10.0.0.99	10.0.0.73	TCP	222	8890 → 58294 [PSH, ACK] Seq=1 Ack=381 Win=64896 Len=156 Tsv=3766626267 TSecr=3563093154 [TCP PDU reassembled in 2650]
2650	395.931494123	10.0.0.99	10.0.0.73	HTTP	1072	HTTP/1.0 200 OK (text/html)
2651	395.931494123	10.0.0.99	10.0.0.73	TCP	66	58294 → 8890 [ACK] Seq=381 Ack=157 Win=64128 Len=0 Tsv=3563093157 TSecr=3766626267
2652	395.931674452	10.0.0.99	10.0.0.73	TCP	66	58294 → 8890 [FIN, ACK] Seq=381 Ack=157 Win=64128 Len=0 Tsv=3563093158 TSecr=3766626268
2653	395.931782736	10.0.0.99	10.0.0.73	TCP	66	58294 → 8890 [FIN, ACK] Seq=381 Ack=157 Win=64128 Len=0 Tsv=3563093158 TSecr=3766626268
2654	395.931782736	10.0.0.99	10.0.0.73	TCP	66	8890 → 58294 [ACK] Seq=1164 Ack=382 Win=64896 Len=0 Tsv=3766632719 TSecr=3563093159
2655	395.931782736	10.0.0.99	10.0.0.73	TCP	74	58304 → 8890 [SYN] Seq=0 Win=64248 Len=0 MSS=1460 SACK_PERM Tsv=3563093158 TSecr=0 WS=128
2656	395.931782736	10.0.0.99	10.0.0.73	TCP	74	8890 → 58304 [SYN, ACK] Seq=1 Ack=1 Win=64256 Len=0 MSS=1460 SACK_PERM Tsv=3766626411 TSecr=3563093159 WS=128
2657	395.931782736	10.0.0.99	10.0.0.73	TCP	60	58304 → 8890 [ACK] Seq=1 Ack=1 Win=64256 Len=0 Tsv=3563093160 TSecr=3766626411
2658	395.931782736	10.0.0.99	10.0.0.73	HTTP	417	GET /favicon.ico HTTP/1.1
2659	395.931782736	10.0.0.99	10.0.0.73	TCP	60	8890 → 58304 [ACK] Seq=1 Ack=352 Win=64896 Len=0 Tsv=3766626413 TSecr=3563093362
2660	395.931782736	10.0.0.99	10.0.0.73	TCP	251	8890 → 58304 [PSH, ACK] Seq=1 Ack=352 Win=64896 Len=185 Tsv=3766626420 TSecr=3563093362 [TCP PDU reassembled in 2661]
2661	395.931782736	10.0.0.99	10.0.0.73	HTTP	481	HTTP/1.0 404 File not found (text/html)
2662	395.931782736	10.0.0.99	10.0.0.73	TCP	60	58304 → 8890 [ACK] Seq=352 Ack=186 Win=64128 Len=0 Tsv=3563093310 TSecr=3766626420
2663	395.931782736	10.0.0.99	10.0.0.73	TCP	66	58304 → 8890 [FIN, ACK] Seq=352 Ack=522 Win=64128 Len=0 Tsv=3563093310 TSecr=3766626420
2664	395.931782736	10.0.0.99	10.0.0.73	TCP	66	8890 → 58304 [ACK] Seq=522 Ack=353 Win=64896 Len=0 Tsv=3766626422 TSecr=3563093310

The above image shows us the wireshark packets of the normal session.

No.	Time	Source	Destination	Protocol	Length	Info
34	19.44899486	10.0.0.99	10.0.0.73	TCP	56	47734 → 109 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
35	19.449207435	10.0.0.99	10.0.0.73	TCP	56	47734 → 554 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
36	19.44844063	10.0.0.99	10.0.0.73	TCP	60	159 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
37	19.449333230	10.0.0.99	10.0.0.73	TCP	56	47734 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
38	19.449781363	10.0.0.99	10.0.0.73	TCP	56	47734 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
39	19.449784277	10.0.0.99	10.0.0.73	TCP	60	554 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
40	19.450216477	10.0.0.99	10.0.0.73	TCP	60	587 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
41	19.450216497	10.0.0.99	10.0.0.73	TCP	60	53 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
42	19.450299609	10.0.0.99	10.0.0.73	TCP	56	47734 → 258 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
43	19.45039253	10.0.0.99	10.0.0.73	TCP	56	47734 → 111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
44	19.450750990	10.0.0.99	10.0.0.73	TCP	60	258 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
45	19.450751735	10.0.0.99	10.0.0.73	TCP	56	47734 → 1123 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
46	19.451594239	10.0.0.99	10.0.0.73	TCP	56	47734 → 8888 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
47	19.451594239	10.0.0.99	10.0.0.73	TCP	60	111 → 47734 [ACK] Seq=1 Ack=1 Win=0 Len=0
48	19.451244185	10.0.0.99	10.0.0.73	TCP	60	1723 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
49	19.451634031	10.0.0.99	10.0.0.73	TCP	56	47734 → 1720 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
50	19.45192976	10.0.0.99	10.0.0.73	TCP	60	8888 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
51	19.452991458	10.0.0.99	10.0.0.73	TCP	56	47734 → 23 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
52	19.452929174	10.0.0.99	10.0.0.73	TCP	60	1720 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
53	19.45273465	10.0.0.99	10.0.0.73	TCP	60	23 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
54	19.452877609	10.0.0.99	10.0.0.73	TCP	56	47734 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
55	19.453185187	10.0.0.99	10.0.0.73	TCP	56	47734 → 993 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
56	19.453299612	10.0.0.99	10.0.0.73	TCP	60	445 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
57	19.453366684	10.0.0.99	10.0.0.73	TCP	56	47734 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58	19.453621113	10.0.0.99	10.0.0.73	TCP	56	47734 → 1025 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
59	19.453797339	10.0.0.99	10.0.0.73	TCP	60	993 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
60	19.454032626	10.0.0.99	10.0.0.73	TCP	56	47734 → 47734 [SYN, ACK] Seq=0 Ack=1 Win=64248 Len=0 MSS=1460
61	19.454063888	10.0.0.99	10.0.0.73	TCP	54	47734 → 22 [RST] Seq=1 Win=0 Len=0
62	19.454506686	10.0.0.73	10.0.0.99	TCP	60	1025 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
63	19.454506686	10.0.0.73	10.0.0.99	TCP	56	47734 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
64	19.454807774	10.0.0.99	10.0.0.73	TCP	56	47734 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
65	19.454807774	10.0.0.99	10.0.0.73	TCP	60	443 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
66	19.455081489	10.0.0.99	10.0.0.73	TCP	56	47734 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
67	19.455284278	10.0.0.73	10.0.0.99	TCP	60	443 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
68	19.455721319	10.0.0.73	10.0.0.99	TCP	60	143 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
69	19.455797908	10.0.0.99	10.0.0.73	TCP	56	47734 → 3386 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
70	19.456006697	10.0.0.73	10.0.0.99	TCP	60	3386 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
71	19.456041987	10.0.0.99	10.0.0.73	TCP	56	47734 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
72	19.456099533	10.0.0.99	10.0.0.73	TCP	56	47734 → 5988 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
73	19.456099533	10.0.0.99	10.0.0.73	TCP	60	443 → 47734 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
74	19.456095133	10.0.0.99	10.0.0.73	TCP	56	47734 → 113 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

This above image shows us the wireshark packets of nmap session.

Conclusion:

In this assignment, we explored the process and impact of network mapping using tools like Nmap, Zenmap, and Wireshark. Using a Kali Linux virtual machine for scanning and an Ubuntu Linux virtual machine as the target, both systems were placed on the same network using Bridged Adapter mode. On the Ubuntu VM, we disabled the firewall and enabled services such as SSH, Netcat listener on port 31337, and a Python webserver on port 8000. From Kali, we first performed normal connections to these services and captured the traffic using Wireshark. Then, we used Nmap and Zenmap to perform several network scans and captured those packets as well for comparison. This allowed us to observe clear differences between typical user behavior and aggressive scanning behavior by network mapping tools. Network mapping software, like Nmap, plays a major role in cybersecurity. These tools are essential for administrators to identify and secure open ports, running services, and potential vulnerabilities within a network. On the positive side, it allows security professionals to identify open ports, active devices, and running services within a network. This helps in detecting vulnerabilities, verifying firewall configurations, and ensuring that only authorized services are exposed. Regular internal scans using tools like Nmap can improve overall security by uncovering weak points before attackers do. However, these same tools can also be used by attackers to discover weaknesses before launching an attack. Our packet analysis in Wireshark showed that normal connections are usually minimal and focused, whereas Nmap scans generate a high volume of SYN packets across multiple ports very quickly. These patterns are easily detectable using tools like Wireshark or Intrusion detection systems (IDS). To protect against malicious scans, network defenders can use firewalls, IDS, traffic rate limiting, and monitoring for unusual port activity. While tools like Nmap and Zenmap provide critical insights for securing services, they also expose networks to risk if used by unauthorized users. Understanding how to use these tools and how to recognize their signatures in packet captures is key to both offensive and defensive cybersecurity.